

Software Architecture — Services and Responsibilities

1. Architectural Principles

The architecture is based on the following principles:

- **Separation of responsibilities**
 - **Modular design**
 - **Non-circular dependencies**
 - **Clear separation between business logic and technical services**
 - **Clear separation between business logic and hardware control**
 - **Event-based communication between components where appropriate**
 - **The orchestrator coordinates communication without making business decisions**
-

2. Application — Orchestration

main_app.c

Role: Main system orchestrator

Responsibilities:

- Initialize the different subsystems.
- Start the display system.
- Start Wi-Fi.
- Initialize the file system.
- Initialize the required hardware.
- Create and manage the tasks required by the system.
- Retrieve data produced by services.
- Transport and distribute events to the appropriate services.
- Coordinate communication between the different parts of the system.

Principle

main_app does not contain phase-selection or alarm business logic.

It orchestrates:

Producer
↓
main_app
↓
Consumer

It acts as a **coordinator**, not as a business decision-maker.

3. Business Services — Decision and Product Logic

voltage_control.c

Role: Central business service for voltage processing and phase selection

Responsibilities:

Step 1 — Numeric validation

Receives the three measured voltages and checks their numerical validity:

NaN → Invalid value
Number → Usable value

Step 2 — Voltage range analysis

Determines the state of each phase:

NORMAL
UNDERVOLTAGE
OVERVOLTAGE
INVALID

Step 3 — Phase qualification

Invalid or out-of-range phases are excluded from the selection process.

Step 4 — Phase selection

Compares the eligible phases and determines the best phase.

Step 5 — Event publication

Publishes relevant events, for example:

VOLTAGE_NORMAL
UNDERVOLTAGE
OVERVOLTAGE
SENSOR_ERROR
PHASE_SELECTED

Events may carry the information required for further processing:

SENSOR_ERROR
├── phase
├── error type
└── attempt count

Principle

`voltage_control` makes business decisions but never directly controls hardware.

Automatic Phase Selection

Automatic phase selection is part of the business logic implemented by `voltage_control`.

It answers:

Which eligible phase should be selected based on the measured voltages and the system's rules?

Manual Phase Selection

Manual phase selection is also a **business-level function**.

It receives a phase-selection request and applies the required rules before producing the corresponding event.

It does not directly control GPIOs or relays.

4. Signaling and Fault Management

alarm_system.c

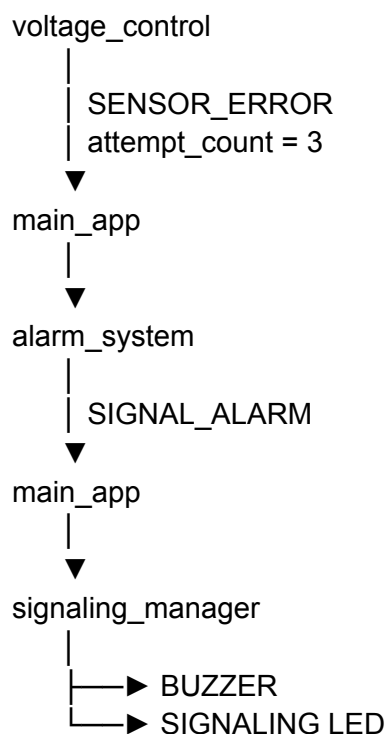
Role: Business service for anomaly and alarm management

`alarm_system` is responsible for interpreting error information and deciding whether and how an anomaly should be signaled.

Responsibilities:

- Receive anomaly events forwarded by the orchestrator.
- Interpret error codes and associated information.
- Determine whether an anomaly should actually trigger an alarm.
- Apply confirmation rules when multiple attempts are required.
- Determine the appropriate signaling command according to the current situation.
- Publish a signaling command for the signaling actuator manager.
- Activate or deactivate alarm signaling at the logical level.

Example



`alarm_system` does not measure voltage and does not directly control the GPIO of the buzzer or signaling LEDs.

Principle

`alarm_system` makes the business decision about signaling but does not control the physical signaling actuators.

It answers:

"What does this error mean, and should it be signaled?"

It does not answer:

"Which GPIO should be activated to produce this signal?"

That responsibility belongs to `signaling_manager`.

5. Actuator Management

`relay_manager.c`

Role: Actuator Manager

`relay_manager` provides the intermediate layer between business decisions and physical relay control.

Responsibilities:

- Receive command events forwarded by the orchestrator.
- Directly execute the decision already made by the business service.
- Translate functional commands into actuator actions.
- Make no phase-selection decisions.

Example:

```
PHASE_SELECTED(L2)
  ↓
main_app
  ↓
relay_manager
  ↓
ACTIVATE RELAY L2
DEACTIVATE RELAY L1/L3
```

Fundamental principle

`relay_manager` does not decide which phase should be selected.

It executes:

"Phase L2 has been selected."

It does not decide:

"I think L2 is the best phase."

signaling_manager.c

Role: Signaling Actuator Manager

`signaling_manager` provides the intermediate layer between the signaling decision made by the business service and the physical signaling actuators.

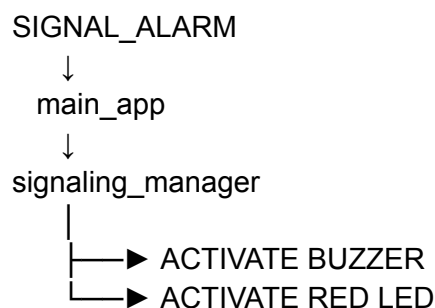
It is responsible for controlling signaling actuators such as:

- Buzzer
- Red signaling LED
- Yellow signaling LED
- Green signaling LED
- Other signaling devices if required

Responsibilities:

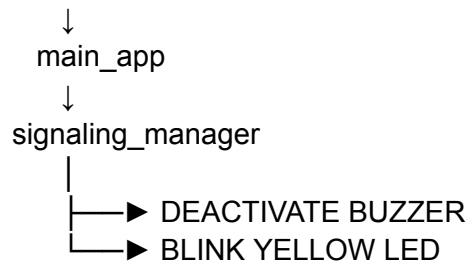
- Receive signaling command events forwarded by the orchestrator.
- Directly execute the signaling decision already made by `alarm_system`.
- Translate functional signaling commands into physical actuator actions.
- Control the buzzer and signaling LEDs through the appropriate PAL.
- Make no decisions about the meaning or severity of errors.
- Contain no alarm business logic.

Example



Another example:

SIGNAL_WARNING



Fundamental principle

`signaling_manager` does not decide why a signal should be activated.

It executes:

"Activate the alarm signaling."

It does not decide:

"This error is serious enough to trigger an alarm."

That decision belongs to `alarm_system`.

6. Technical Services — Acquisition and System Infrastructure

`voltage_sensor.c`

Role: Voltage measurement service

Primary responsibility:

Measure the three phase voltages and transmit the measured values.

It:

- Measures L1.
- Measures L2.
- Measures L3.
- Transmits the measurements.
- Does not interpret them.
- Does not decide which phase should be selected.

Flow:

Hardware
↓
voltage_sensor
↓
Voltage measurements
↓
voltage_control

Principle

The sensor answers:

"What is the measured value?"

It does not answer:

"What should the system do with this value?"

7. Actuator / Hardware Abstraction

actuators_pal.c

Role: Actuator PAL

Responsibilities:

- Provide a hardware-independent interface for controlling actuators.
- Receive commands from actuator managers.
- Hide hardware-specific implementation details from higher-level software.

Example:

relay_manager
↓
actuator PAL
↓
GPIO driver

The same principle applies to the signaling actuators:

signaling_manager
↓
actuator PAL

↓
GPIO driver

The PAL therefore acts as a boundary between the software and the underlying hardware implementation.

gpio_drivers_pal.c

Role: GPIO driver abstraction

Responsibilities:

- Provide the GPIO operations required by the PAL.
 - Hide microcontroller-specific GPIO implementation details.
 - Minimize direct hardware dependencies.
-

8. Middleware / System Infrastructure

Wi-Fi

Role: Network connectivity service

Responsibilities:

- Initialize Wi-Fi.
 - Manage the network connection.
 - Manage connectivity status.
 - Provide network connectivity to the rest of the system.
-

Web Server

Role: Web communication interface

Responsibilities:

- Handle HTTP requests.
- Provide system data to the Web UI.
- Receive commands from the Web UI.

- Convert Web requests into information or events that can be processed by the application.

The Web Server must not bypass the business logic to directly control hardware.

Web UI



Web Server



Application / Business Service



Decision



Actuator Manager

File System

Role: Local storage service

Responsibilities:

- Initialize the file system.
- Read required data.
- Write data provided by the application.
- Provide storage capabilities to the rest of the system.

The File System receives the data that `main_app` provides to it.

Display System

Role: Local display service

Responsibilities:

- Initialize the display.
- Receive information to be displayed.
- Present the system state to the user.

The Display System **does not interpret business rules**.

It receives prepared information from the application:

`main_app`



display_system



LCD / Display

9. Services Grouped by Affinity

Application

main_app.c

Coordinates the system.

Business Services

voltage_control

automatic phase selection

manual phase selection

alarm_system

Make decisions and apply business rules.

Technical Services

voltage_sensor

display_system

Provide technical capabilities to the system.

Middleware / Infrastructure

Wi-Fi

Web Server

File System

Provide the infrastructure required by the application.

Actuator Management

relay_manager
signaling_manager

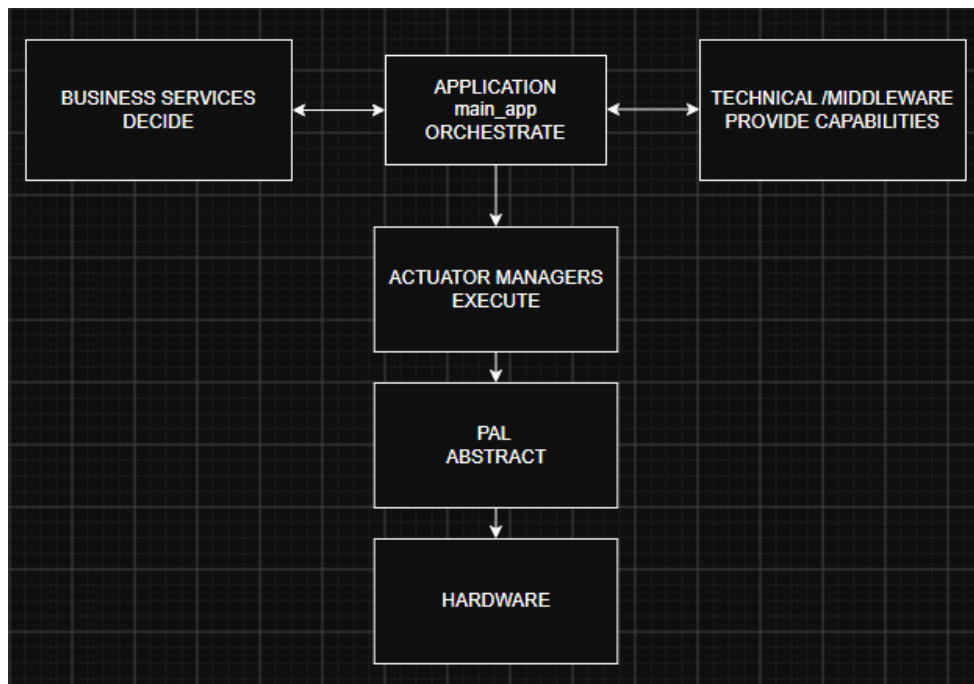
Translate functional decisions into actuator commands without making business decisions.

PAL / Hardware Abstraction

actuators_pal
gpio_drivers_pal

Isolate the software from hardware-specific implementation details.

10. Overall Responsibility Model



Fundamental Rule

Business services decide.

The orchestrator coordinates.

Technical services provide capabilities.

Actuator managers execute.

The PAL abstracts the hardware.

Drivers communicate with the hardware.